Question Bank Management System

A DSA-3 driven Java application for intelligent question retrieval and paper generation

DATA STRUCTURES & ALGORITHMS – 3 | 25CS2103E | JAVA + FILES • NO DATABASE

Team 23 | Section 6
P. Shashe Preetham • J. Advaith • O. Sai Sujith
Guide: Dr. V. Sireesha

1. Project Overview

What the system solves and why it matters

The core idea

Educational institutions accumulate questions across semesters, subjects and difficulty levels. The project turns scattered question files into a structured, searchable question bank using algorithms rather than a database.

Store & organise

Questions are kept in structured text files and categorised by subject, unit, difficulty and course outcome.

Search intelligently

KMP / Rabin–Karp support keyword search; edit distance supports fuzzy retrieval for misspelled queries.

Generate better papers

Questions can be selected against criteria while usage history helps reduce repeated questions.

2. Problem Statement

The academic workflow is simple — the data problem is not

Today’s pain points

• Questions are scattered across files, folders and printed sheets.
• Finding a suitable question requires repeated manual searching.
• Similar or repeated questions can enter papers.
• Difficulty / unit / CO balance is hard to maintain manually.
• There is little visibility into previous question usage.

What this causes

• Higher preparation time
• Repeated questions
• Inconsistent paper quality
• Poor reuse of existing question material
• More effort for faculty during assessment design

The DSA opportunity

Question banks are fundamentally text-search and optimisation problems.

Instead of making the project a CRUD + database application, we make the algorithms the main product.

3. Existing Solution vs Our Implementation

Moving from storage-first systems to algorithm-first search

Typical / Existing Approach

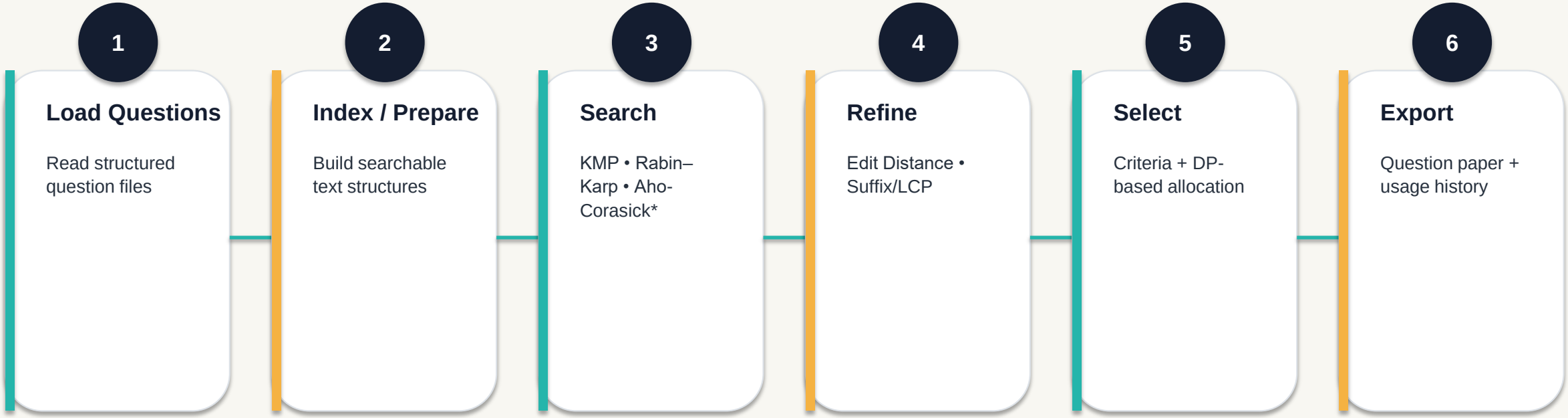
- Database-centric CRUD application
- Basic keyword / linear search
- Manual filtering and paper preparation
- Separate files often treated as independent data
- Limited fuzzy or similarity search
- Question reuse checked manually

Our DSA-3 Implementation

- Java application with structured file storage
- KMP / Rabin–Karp for efficient text search
- Edit Distance for fuzzy query matching
- Suffix Array + LCP for similarity / duplicate analysis
- Algorithm-driven selection for paper generation
- Usage history stored and processed from files

4. Proposed System

A lightweight, file-backed Java system where DSA performs the heavy work



* Aho–Corasick can be included when the project needs multi-keyword / multi-pattern matching; the core build remains intentionally manageable.

5. Objectives

Project goals expressed as measurable DSA work

Fast retrieval

Search a large question file efficiently using advanced string matching.

Fuzzy search

Handle spelling variation using edit-distance based matching.

Similarity

Detect closely related questions using suffix-based text analysis.

Smart selection

Generate a balanced set of questions from constraints.

No database

Keep persistence simple with readable, structured files.

Complexity aware

Compare algorithms using time / space complexity.

6. Why This Is a DSA-3 Project

The project directly uses the advanced algorithm families in the handout

Module 2 • String Algorithms

KMP, Rabin–Karp, Z-function / Aho–Corasick and suffix structures for question retrieval.

Module 3 • Dynamic Programming

Wagner–Fischer edit distance and DP-based constrained question selection.

Module 5 • Complexity & Approximation

Analyse search / selection costs and use approximation ideas if perfect paper balancing becomes expensive.

Module 6 • Randomised Algorithms

Optional enhancement: randomized hashing for robust indexing / collision reduction.

7. Approach Towards the Problem

Design the workflow around queries, not around tables

1. **Input** — Faculty adds a question with subject, unit, difficulty, CO and tags.
2. **Persist** — Question object is serialised into a structured text file.
3. **Search** — Query is sent to a selected string-matching algorithm.
4. **Rank / refine** — Fuzzy and similarity checks improve relevance.
5. **Select** — Criteria are converted into a constrained selection problem.
6. **Record** — Generated-paper usage is appended to history.

8. System Architecture

Java-only application; files are the persistence layer

USER / FACULTY Console or simple Java UI

APPLICATION LAYER Question Manager • Search Engine • Filter Engine • Paper Generator

DSA ENGINE KMP | Rabin–Karp | Edit Distance | Suffix Array + LCP | DP Selection

FILE STORAGE questions.txt • users.txt • usage_history.txt • generated_papers.txt

9. File-Based Backend Design

No MySQL • No MongoDB • No external database

Suggested file organisation

data/
  questions.txt
  users.txt
  usage_history.txt
  generated_papers.txt

Each record contains fixed fields separated by a safe delimiter such as |.
Java File I/O loads the records into custom in-memory structures for algorithmic processing.

Question record

ID | Subject | Unit | Difficulty | CO | Marks | Question Text | Tags

Usage record

Question ID | Paper ID | Date | User | Usage Count

Why files?

Simple deployment • human-readable • easy to demonstrate • keeps the project focused on DSA rather than database engineering.

10. Core DSA Algorithms

The algorithms are the differentiator — not just implementation details

KMP

Exact keyword search
O(n + m)

Best for repeated searches with the same pattern.

Rabin–Karp

Rolling hash
Average-case O(n + m)

Useful for hash-assisted search and multiple query checks.

Edit Distance

Wagner–Fischer
O(nm)

Fuzzy matching for spelling variation.

Suffix + LCP

Similarity analysis
Depends on construction/search

Find related question text and common substrings.

DP Selection

Constraint-aware selection

Choose questions while tracking marks / unit / difficulty / CO targets.

Randomised Hashing*

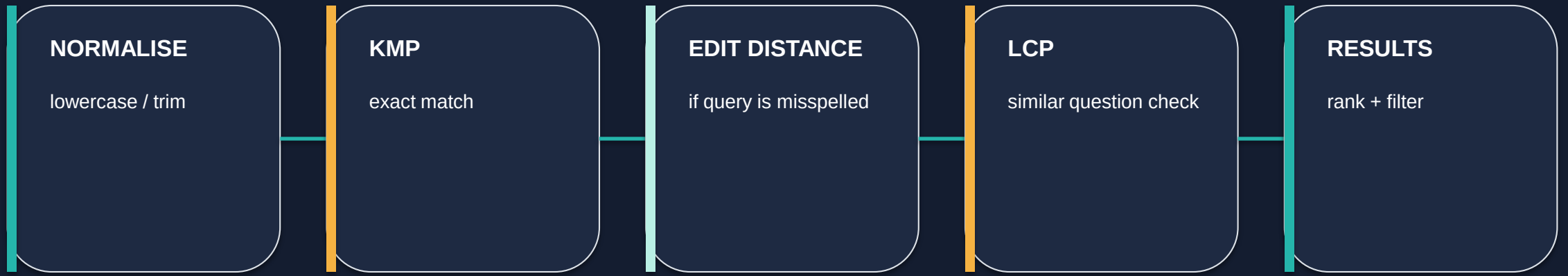
Collision reduction

Optional advanced extension aligned with Module 6.

11. Search Example

From a teacher’s query to an algorithmic result

```
Query:  "knuth morris pratt algorithm"
```



The teacher sees relevant questions; the presentation demonstrates why an advanced algorithm was chosen.

12. Main Functional Modules

Keep the feature set practical while keeping DSA visible

Question Management

Add / edit / remove / view questions in files.

Advanced Search

Exact and multi-pattern keyword retrieval.

Fuzzy Search

Suggest close matches for misspelled queries.

Similarity Check

Flag highly similar or repeated questions.

Smart Filters

Subject • unit • difficulty • CO • marks.

Paper Generator

Generate a paper from selected constraints.

Usage Tracker

Record questions used in generated papers.

Role Access

Admin and faculty permissions using file records.

13. Complexity & Performance

The project should prove algorithmic improvement, not just functionality

Technique	Typical cost	Project role
Naïve matching	O(nm)	Baseline / comparison
KMP	O(n + m)	Primary exact search
Rabin–Karp	Average O(n + m)	Hash-assisted search
Edit Distance	O(nm)	Fuzzy matching
Suffix Array + LCP	Construction-dependent	Similarity / duplicate analysis
DP selection	Problem-dependent	Constraint-based paper generation

Demo idea: compare the number of character comparisons / search time for Naïve vs KMP on the same question bank.

14. Expected Outcomes

What the finished system should demonstrate

- A searchable, structured question bank stored entirely in files.
- Noticeably faster exact search than naïve character-by-character matching for suitable inputs.
- Fuzzy retrieval for misspelled or slightly different queries.
- Detection of highly similar / repeated question text.
- Question paper generation based on selected academic criteria.
- Usage history that helps reduce repeated questions over time.
- Clear complexity comparison of the algorithms used.

Primary value

Less manual searching + more consistent assessment preparation.

Academic value

Every major feature can be explained through DSA-3 concepts.

Engineering value

Simple deployment: Java runtime + data files.

15. Scope, Limitations & Future Scope

A realistic academic project — strong in algorithms, intentionally light on infrastructure

Current scope

- Java application
- File-based persistence
- Advanced text search
- Fuzzy matching
- Similarity detection
- Filters and question metadata
- Paper generation
- Usage tracking

Limitations

- File storage is not ideal for concurrent multi-user production systems.
- Large-scale indexing may require careful memory management.
- Perfectly balanced paper generation can become computationally expensive.
- Authentication remains lightweight without a database.

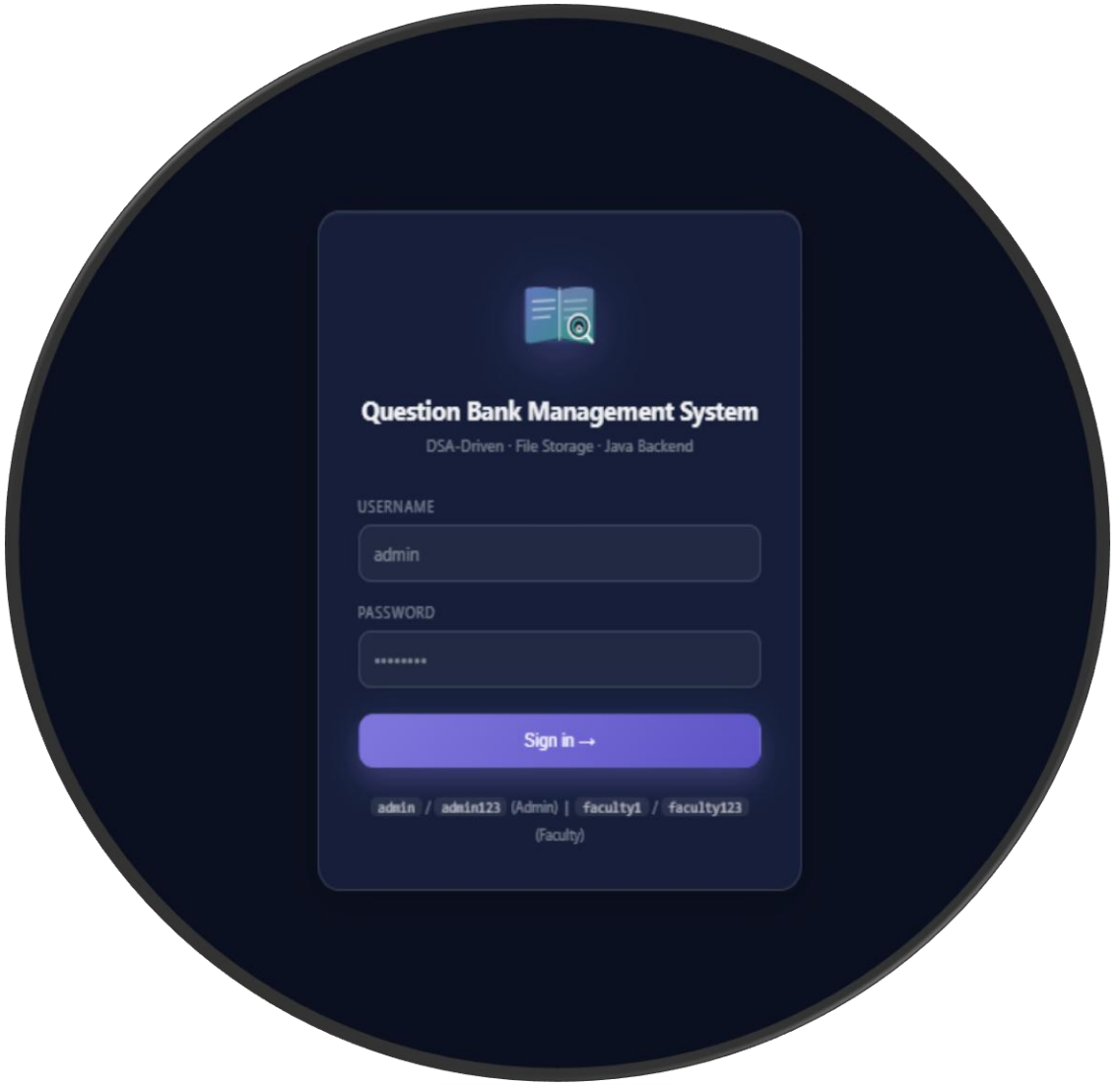
Future scope

- Persistent database / web deployment as a separate engineering layer.
- Parallel search for very large repositories.
- Stronger randomized hashing.
- Network-flow based allocation for complex paper constraints.
- Analytics dashboard for question usage and quality.

CODE&OUTPUT

QBMS-Frontend > index.html

```
8   <style>
52    .blob-a { width: 320px; height: 320px; background: var(--accent-1); top: -80px; left: -60px; }
53    .blob-b { width: 280px; height: 280px; background: var(--accent-2); bottom: 10%; right: -80px; animation-delay: 4s; }
54    .blob-c { width: 220px; height: 220px; background: var(--accent-3); bottom: -60px; left: 30%; animation-delay: 8s; }
55    @keyframes float {
56      0%, 100% { transform: translate(0,0) scale(1); }
57      33% { transform: translate(30px,-20px) scale(1.06); }
58      66% { transform: translate(-20px,25px) scale(0.96); }
59    }
60    .glass {
61      background: var(--glass-bg); border: 1px solid var(--glass-border);
62      backdrop-filter: blur(24px) saturate(180%); -webkit-backdrop-filter: blur(24px) saturate(180%);
63      border-radius: var(--radius);
64      box-shadow: 0 8px 32px rgba(0,0,0,0.18), inset 0 1px 0 rgba(255,255,255,0.08);
65      transition: transform 0.35s cubic-bezier(.2,.8,.2,1), box-shadow 0.35s ease, border-color 0.35s ease;
66    }
67    .glass:hover { border-color: var(--glass-hi); }
68
69    /* ===== LOGIN ===== */
70    #loginScreen {
71      min-height: 100vh; display: flex; align-items: center;
72      justify-content: center; padding: 24px; position: relative;
73    }
74    .login-card { width: 100%; max-width: 400px; padding: 36px 32px 28px; animation: rise 0.6s cubic-bezier(.2,.8,.2,1) both; }
75    .login-logo { display: flex; align-items: center; justify-content: center; margin-bottom: 20px; }
76    .login-logo svg { width: 72px; height: 72px; filter: drop-shadow(0 8px 20px rgba(127,119,221,0.5)); }
77    .login-title { text-align: center; font-size: 20px; font-weight: 700; margin-bottom: 4px; letter-spacing: -0.02em; }
78    .login-sub { text-align: center; font-size: 12.5px; color: var(--text-low); margin-bottom: 28px; }
79    .login-field { margin-bottom: 14px; }
80    .login-field label { font-size: 11.5px; color: var(--text-low); text-transform: uppercase; letter-spacing: 0.05em; display: block; mar
81    .login-error {
82      display: none; font-size: 12.5px; color: var(--danger);
83      background: rgba(226,75,74,0.12); border: 1px solid rgba(226,75,74,0.35);
84      border-radius: 10px; padding: 9px 12px; margin-bottom: 14px; animation: shake 0.4s ease;
85    }
86    .login-error.show { display: block; }
87    @keyframes shake {
88      0%, 100% { transform: translateX(0); } 25% { transform: translateX(-6px); } 75% { transform: translateX(6px); }
89    }
90    .login-hint { text-align: center; font-size: 11px; color: var(--text-low); margin-top: 18px; line-height: 1.8; }
91    .login-hint code { background: rgba(255,255,255,0.08); padding: 1px 6px; border-radius: 5px; color: var(--text-mid); }
```



CODE

QuestionBankManagementSystem > src > algorithms > AhoCorasick.java

```java
package algorithms;

import structures.CustomArrayList;

/*
 * Algorithm: Aho-Corasick multi-pattern string matching automaton
 * Complexity: O(n + Σ|patterns| + total_matches)
 *
 * How it works:
 *  1. Build a Trie from all pattern keywords.
 *  2. Compute failure links via BFS (like KMP failure function but on a Trie).
 *  3. Scan the text once; follow goto/failure transitions to find all patterns
 *     simultaneously in a single pass.
 *
 * Returns the number of keyword matches found in the text.
 */
public class AhoCorasick {

    // ---- Trie node ----
    private static class Node {
        int[] children;
        int failLink;
        boolean isEnd;
        String keyword;

        Node() {
            children = new int[26];
            java.util.Arrays.fill(children, -1);
            failLink = 0;
            isEnd = false;
            keyword = null;
        }
    }

    private CustomArrayList<Node> trie;
    private long comparisons;
    private boolean built;

    public AhoCorasick() {
        trie = new CustomArrayList<>();
        trie.add(new Node()); // root = node 0
```

```java
package model;

import structures.CustomArrayList;

/*
 * Model: SearchResult
 * Purpose: Holds the outcome of running a search/matching algorithm so the
 * UI and PerformanceAnalyzer (later phases) can display a uniform report:
 *   Algorithm | Query | Matches | Comparisons | Execution Time | Complexity
 * Not persisted to file - this is an in-memory reporting object only.
 */
public class SearchResult {

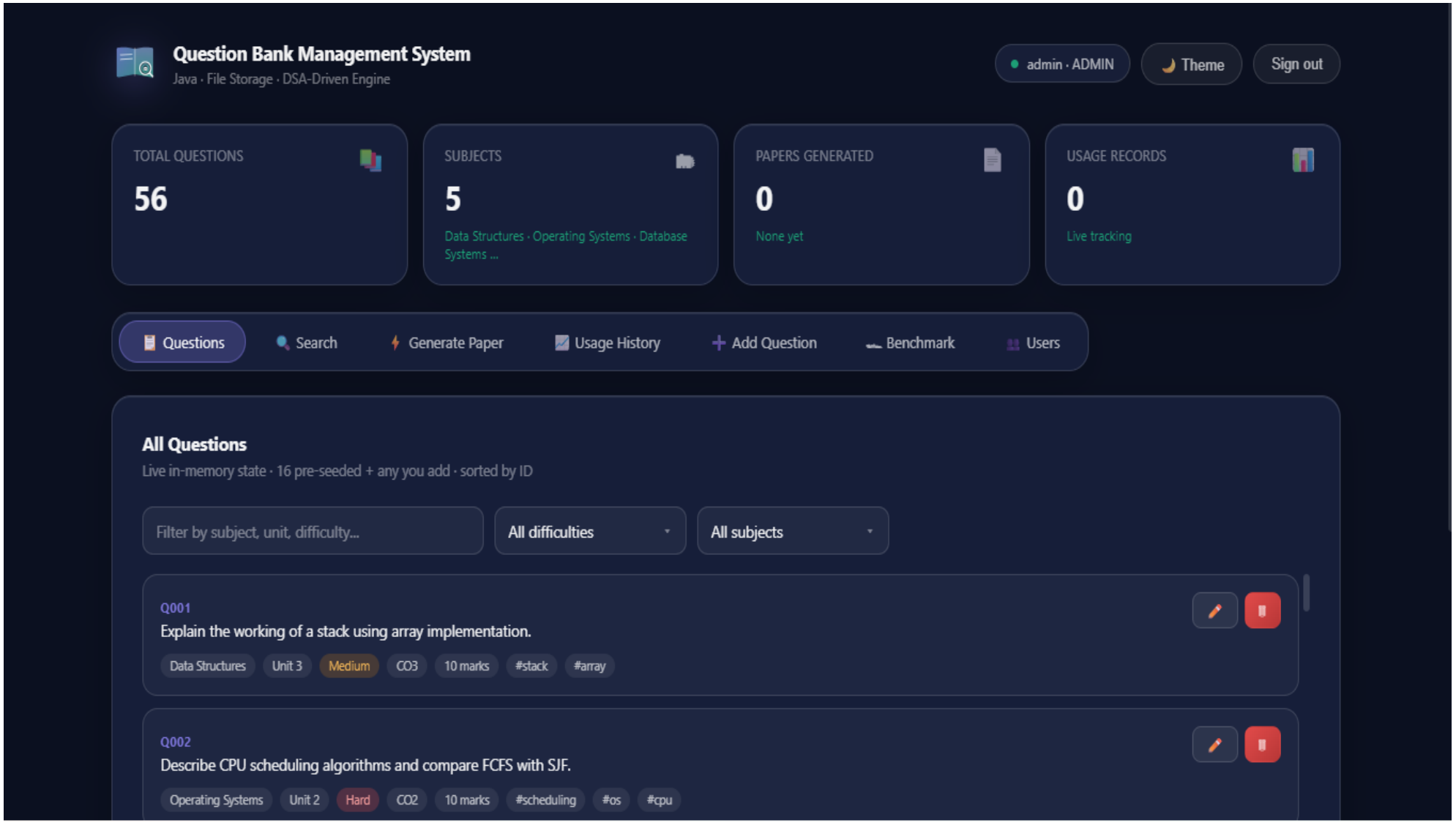
    private String algorithmName;
    private String query;
    private CustomArrayList<String> matchedQuestionIds;
    private long comparisons;
    private double executionTimeMs;
    private String complexity;

    public SearchResult(String algorithmName, String query) {
        this.algorithmName = algorithmName;
        this.query = query;
        this.matchedQuestionIds = new CustomArrayList<>();
        this.comparisons = 0;
        this.executionTimeMs = 0.0;
        this.complexity = "";
    }

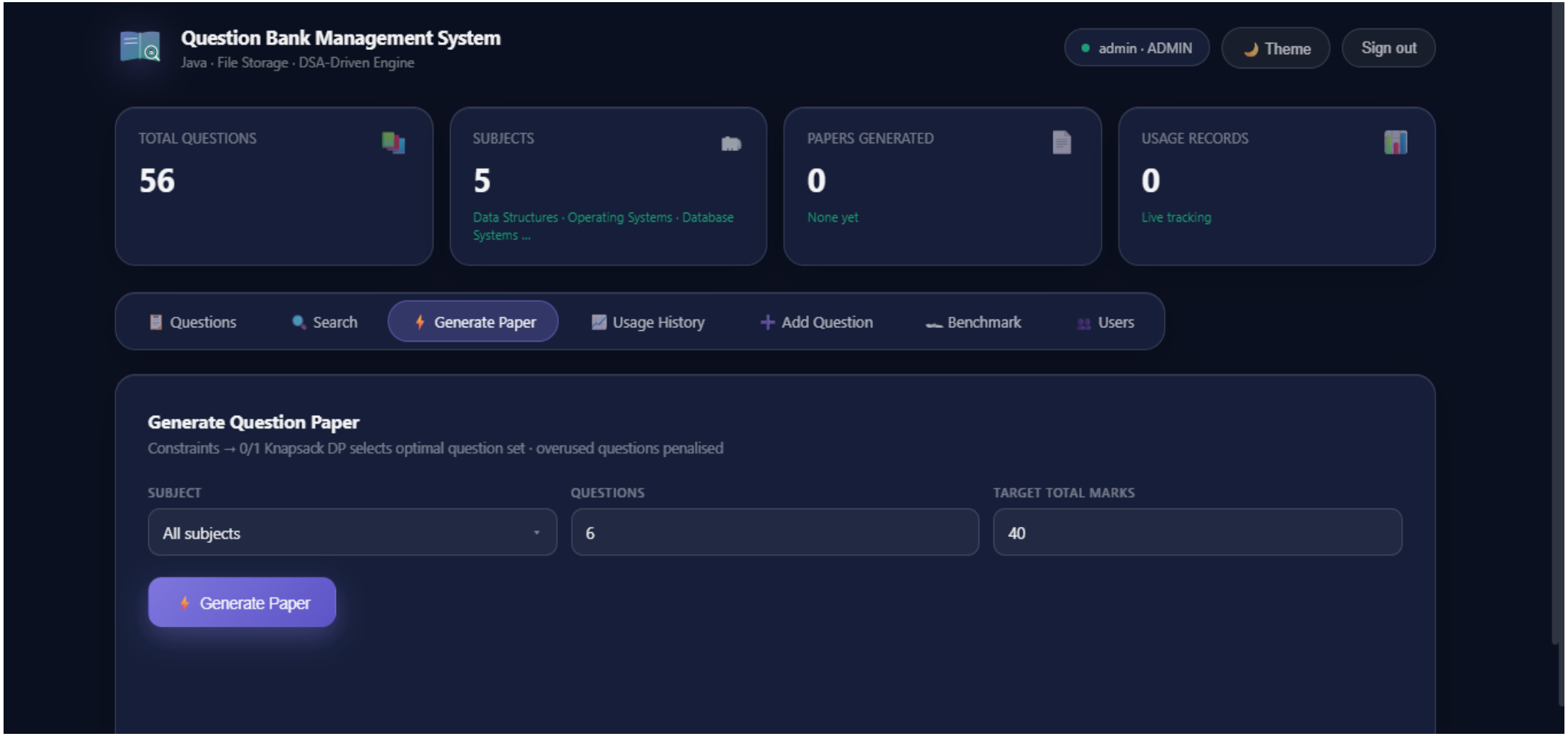
    public String getAlgorithmName() { return algorithmName; }
    public String getQuery() { return query; }
    public CustomArrayList<String> getMatchedQuestionIds() { return matchedQuestionIds; }
    public long getComparisons() { return comparisons; }
    public double getExecutionTimeMs() { return executionTimeMs; }
    public String getComplexity() { return complexity; }

    public void addMatch(String questionId) { matchedQuestionIds.add(questionId); }
    public void setComparisons(long comparisons) { this.comparisons = comparisons; }
    public void setExecutionTimeMs(double executionTimeMs) { this.executionTimeMs = executionTimeMs; }
    public void setComplexity(String complexity) { this.complexity = complexity; }
```

OUTPUT


Question Bank Management System
Java · File Storage · DSA-Driven Engine
admin · ADMIN
Theme
Sign out
TOTAL QUESTIONS
56
SUBJECTS
5
Data Structures · Operating Systems · Database Systems ...
PAPERS GENERATED
0
None yet
USAGE RECORDS
0
Live tracking
Questions
Search
Generate Paper
Usage History
Add Question
Benchmark
Users
All Questions
Live in-memory state · 16 pre-seeded + any you add · sorted by ID
Filter by subject, unit, difficulty...
All difficulties
All subjects
Q001
Explain the working of a stack using array implementation.
Data Structures
Unit 3
Medium
CO3
10 marks
#stack
#array
Q002
Describe CPU scheduling algorithms and compare FCFS with SJF.
Operating Systems
Unit 2
Hard
CO2
10 marks
#scheduling
#os
#cpu

OUTPUT


Question Bank Management System
Java · File Storage · DSA-Driven Engine
admin · ADMIN
Theme
Sign out
TOTAL QUESTIONS
56
SUBJECTS
5
Data Structures · Operating Systems · Database Systems ...
PAPERS GENERATED
0
None yet
USAGE RECORDS
0
Live tracking
Questions
Search
Generate Paper
Usage History
Add Question
Benchmark
Users
Generate Question Paper
Constraints → 0/1 Knapsack DP selects optimal question set · overused questions penalised
SUBJECT
All subjects
QUESTIONS
6
TARGET TOTAL MARKS
40
Generate Paper

Conclusion

n Bank Management System turns a common academic workflow into a practical DSA-3 p

Java handles the application. Files handle persistence. Advanced algorithms handle the intelligence.

STRING SEARCH

FUZZY MATCHING

SIMILARITY

SMART SELECTION

Thank You • Questions & Discussion